

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Функциональное программирование

**Отчет о выполнении лабораторных
работ 1-5**

Студент,
группы 5130201/40003

_____ Лебедев К. О.

Преподаватель,
к.т.н.

_____ Моторин Д.Е.

«_____» _____ 20__ г.

Санкт-Петербург, 2026

Содержание

1	Введение	3
2	Временная диаграмма выполнения заданий	3
2.1	Пояснения к диаграмме	3
2.2	Временная диаграмма	4
3	Структура и содержание репозитория	4
3.1	Общая структура	4
3.2	Содержание разделов	5
4	Лабораторная работа №1	5
4.1	Коммиты и выполняемая работа	5
4.2	Вопросы защиты и их решения	7
5	Лабораторная работа №2	7
5.1	Часть 1. Тип MyEvolution	7
5.2	Часть 2. Типы MyMaybe, MyEither, MyTree	8
5.3	Часть 3. Парсеры	9
5.4	Вопросы защиты и их решения	9
6	Лабораторная работа №3	10
6.1	Часть 1. Глитч-эффекты	10
6.2	Часть 2. Лабиринт на монаде RWS	11
6.3	Вопросы защиты и их решения	11
7	Лабораторная работа №4	11
7.1	Коммиты и выполняемая работа	12
7.2	Вопросы защиты и ответы на них	12
8	Лабораторная работа №5	13
8.1	Коммиты и выполняемая работа	13
8.2	Задание преподавателя	14
A	Приложение A. Листинги кода	16
A.1	Лабораторная работа №1	16
A.2	Лабораторная работа №2	19
A.3	Лабораторная работа №3	25
A.4	Лабораторная работа №4	29
A.5	Лабораторная работа №5	32

1 Введение

В данном отчёте описывается процесс выполнения лабораторных работ 1–5 по дисциплине «Функциональное программирование». Все работы выполнены на языке Haskell. Для каждой работы приводится перечень коммитов с описанием внесённых изменений, рассмотренные при защите вопросы и их решения. Полные листинги исходного кода вынесены в Приложение А.

Исходный код размещён в репозитории на GitHub с именем `hask_pr01`. Каждая работа находится в отдельном каталоге с именем `practice_` и номером работы (например, `practice_01`).

2 Временная диаграмма выполнения заданий

2.1 Пояснения к диаграмме

На рисунке 1 приведены этапы выполнения лабораторных работ с указанием диапазона дат активной работы (по датам коммитов). Работа над заданиями велась последовательно с марта по май 2026 года.

2.2 Временная диаграмма

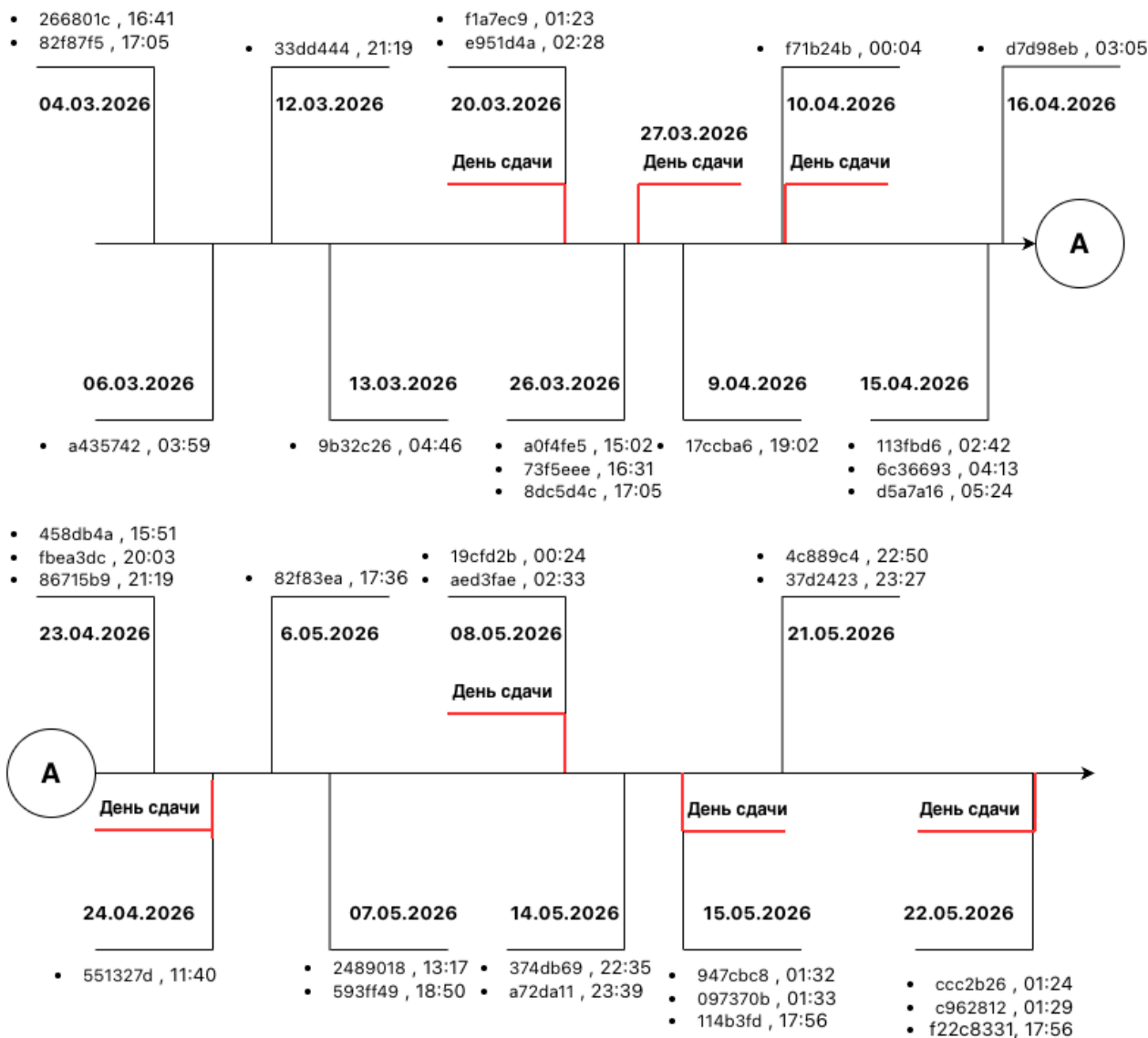


Рис. 1: Временная диаграмма выполнения лабораторных работ

3 Структура и содержание репозитория

3.1 Общая структура

Репозиторий `has_k_pr01` организован по каталогам с именем `practice_` и номером работы. В каждом каталоге содержатся только файлы и/или `stack`-проекты, относящиеся к соответствующему заданию.

3.2 Содержание разделов

1. `practice_01` — файлы `pr01_1.hs` и `pr01_2.hs` первой работы.
2. `practice_02/part1/MyEvolProject` — проект первой части второй работы (тип `MyEvolution`).
3. `practice_02/part2/myParser` — stack-проект второй и третьей частей второй работы (модули `MyTypes` и `MyParsers`).
4. `practice_03/myGlithes` — первая часть третьей работы (глитч-эффекты).
5. `practice_03/myMaze` — вторая часть третьей работы (RWS-решатель лабиринта).
6. `practice_04/myMazeGame` — четвёртая работа (игра на трансформерах монад).
7. `practice_05/myFunctionsAndTests` — пятая работа (QuickCheck-тесты).
8. `report` — файл `report.pdf` с отчётом о выполнении работ.

4 Лабораторная работа №1

В первой работе реализованы стандартные функции работы со списками, функции высшего порядка, свёртки, а также система типов для процесса выпекания тортов с контролем ингредиентов.

4.1 Коммиты и выполняемая работа

- **Коммит 1. Инициализация репозитория (04.03.2026, 16:41, 266801c)**
Создан репозиторий на GitHub. Добавлены первоначальные файлы первой части задания.
- **Коммит 2. Добавление `myHead` и `myTail` (04.03.2026, 17:05, 82f87f5)**
Реализованы первые функции: `myHead` и `myTail` — отделение головы и хвоста списка.
- **Коммит 3. Стандартные функции и функции высшего порядка (06.03.2026, 03:59, a435742)**
Реализованы и доработаны функции первой части (`pr01_1.hs`):
 - a) `myHead`, `myTail` — отделение головы и хвоста;
 - b) `myZip`, `myZip3` — попарное и потройное объединение;

- c) `myUnzip` — разделение списка пар на пару списков;
- d) `myFST`, `mySND`, `myTHRD` — проекции кортежа;
- e) функции высшего порядка: `myMap`, `myZipWith`, `myZipWith3`, `myAll`, `myAny`, `myComposition`.

- **Коммит 4. `myZipSave` и `myUnzipSave` (12.03.2026, 21:19, 33dd444)**

Во второй части (`pr01_2.hs`) реализованы:

- a) `myZipSave :: [a] -> [b] -> ([a, b], [a], [b])` — объединение списков с сохранением остатка более длинного;
- b) `myUnzipSave :: ([a, b], [a], [b]) -> ([a], [b])` — обратная операция с восстановлением остатка.

- **Коммит 5. Свёртки и обработка `Maybe` (13.03.2026, 04:46, 9b32c26)**

Реализованы функции на основе свёрток: `myFoldr`, `myFoldl`, `myFoldl1`, `myFoldr1`, `myReverse` (через свёртку), `myTakeWhile` и `mySpan` (через `myFoldr`), а также `myMaybe` — обработка возможно отсутствующего значения.

- **Коммит 6. `myMap` для `MyList` и `myUnfoldr` (20.03.2026, 01:23, f1a7ec9)**

Определён рекурсивный тип `MyList a = MyEmpty | MyCons a (MyList a)`; для него реализована `myMap`. Добавлена развёртка `myUnfoldr :: (b -> Maybe (a, b)) -> b -> [a]`.

- **Коммит 7. Типы для выпекания тортов (20.03.2026, 02:28, e951d4a)**

Определены типы `Ingredient`, `FillingMix`, `Dough`, `CakeDoughType`, `Cake`, `Action`. Каждый конструктор ингредиента хранит количество (`Int`), что позволяет контролировать объём расхода.

- **Коммит 8. Доработка контроля ингредиентов (26.03.2026, 15:02, a0f4fe5)**

Уточнены конструкторы ингредиентов (добавлен параметр `Int`); функция `makeCakeMix` исправлена. Функция `cakeDough` откорректирована, вспомогательная функция проверки количества удалена за счёт переноса проверок в охранные выражения.

- **Коммит 9. Примеры выпекания (26.03.2026, 16:31, 73f5eee)**

Реализованы `chocolateCakeDough`, `vanillaCakeDough`, `carrotCakeDough` и `chocolateCake`, `vanillaCake`, `carrotCake`. Добавлены примеры удачного приготовления трёх видов тортов и примеры ошибочных ситуаций: недостаток ингредиентов, неверные ингредиенты, несоответствие смеси и теста.

- Коммит 10. Реорганизация каталогов (26.03.2026, 17:05, 8dc5d4c)

Файлы перенесены в каталоги `practice_01` и `practice_02`.

4.2 Вопросы защиты и их решения

- Вопрос 1 (задание дано 20.03.2026)

Реализовать функцию типа `Maybe (a -> a) -> [Maybe a] -> [Maybe a]`.

```
1 func :: Maybe (a -> a) -> [Maybe a] -> [Maybe a]
2 func Nothing _ = []
3 func _ [] = []
4 func (Just f) (xs) = map g xs where
5     g (Just x) = Just (f x)
6     g Nothing = Nothing
```

Листинг 1: `func` — применение `Maybe`-функции к списку `Maybe`-значений

- Вопрос 2 (задание дано 27.03.2026)

Реализовать функцию типа `Either String (a -> b -> c) -> Either String [(a,b)] -> Either String [c]`.

```
1 func0 :: Either String (a -> b -> c) -> Either String [(a,b)] -> Either
  ↳ String [c]
2 func0 (Left a) (Left b) = Left (a ++ b)
3 func0 (Left a) _ = Left a
4 func0 _ (Left b) = Left b
5 func0 (Right f) (Right xs) = Right (helper f xs) where
6     helper f ((a,b): xxs) = (f a b) : helper f xxs
7     helper f [] = []
```

Листинг 2: `func0` — применение `Either`-функции к списку пар `Either`

5 Лабораторная работа №2

Вторая работа состоит из трёх частей: реализация классов типов для перечислимого типа `MyEvolution`; собственные типы `MyMaybe`, `MyEither`, `MyTree` как представители стандартных классов типов; парсеры выражений (собственный и на основе библиотек `Parsec` и `Attoparsec`).

5.1 Часть 1. Тип `MyEvolution`

- Коммит 1. Проект `MyEvolProject` и тип `MyEvolution` (09.04.2026, 19:02, 17ccba6)

Создан проект `MyEvolProject`. Определён тип-сумма `MyEvolution` с десятью конструкторами — этапами эволюции (от `LUCA` до `Humans`). Вручную реализованы представители классов:

- a) `Show` — каждый конструктор преобразуется в строку (например, `LUCA` → «Last Universal Common Ancestor»);
- b) `Read` — обратное преобразование строки в конструктор;
- c) `Eq` — равенство одинаковых конструкторов;
- d) `Enum` — `fromEnum/toEnum` для порядковых номеров;
- e) `Ord` — сравнение через `fromEnum`;
- f) `Bounded` — `minBound = LUCA, maxBound = Humans`.

Дополнительно определён тип `MyEvolution'` с теми же конструкторами, представители классов получены автоматически через `deriving`. В `Main.hs` выводится отсортированный список всех значений типа.

5.2 Часть 2. Типы `MyMaybe`, `MyEither`, `MyTree`

- **Коммит 2. `MyTree: Functor` (10.04.2026, 00:04, f71b24b)**

Определён тип `MyTree a = Leaf a | Node a (MyTree a) (MyTree a)`, хранящий значения и в листьях, и в узлах. Реализован представитель `Functor`.

- **Коммит 3. `MyTree: Foldable` (15.04.2026, 02:42, 113fbd6)**

Реализован представитель `Foldable` для `MyTree` (`foldr`, `foldMap`, `length`). Добавлены примеры вызовов в комментариях.

- **Коммит 4. `MyEither` (16.04.2026, 03:05, d7d98eb)**

Определён тип `MyEither e a = MyLeft e | MyRight a`. Реализованы представители `Functor`, `Foldable`, `Applicative` и `Semigroup`. Добавлены примеры вызовов всех реализованных функций (`<>`, `stimes`, `sconcat`, `(<*)`, `(<*)`, `liftA2`, `(<$)`, `fold` и другие).

- **Коммит 5. `MyTree: Applicative` (15.04.2026, 04:13, 6c36693)**

Реализован представитель `Applicative` для `MyTree` (`pure`, `<*>`) и примеры вызовов `<*>`, `liftA2`, `<*>`, `<*`.

- **Коммит 6. `MyMaybe` (15.04.2026, 05:24, d5a7a16)**

Определён тип `MyMaybe a = MyJust a | MyNothing`. Реализованы представители `Functor`, `Applicative`, `Semigroup`, `Monoid` (с `mempty = MyNothing`) и `Foldable`. В комментариях приведены примеры вызовов `fmap`, `<$`, `pure`, `<*>`, `liftA2`, `<>`, `sconcat`, `stimes`, `mappend`, `mconcat`, `foldr`, `foldMap`, `fold`.

5.3 Часть 3. Парсеры

- **Коммит 7. Собственный парсер MyParser (23.04.2026, 15:51, 458db4a)**

Реализован модуль `MyParsers.MyParser`. Определён тип `newtype Parser tok a = Parser { runParser :: [tok] -> MyMaybe ([tok], a) }`, использующий собственный тип `MyMaybe` вместо стандартного `Maybe`. Реализованы представители `Functor`, `Applicative`, `Alternative`, базовые парсеры (`satisfy`, `char`, `lower`, `digit`) и комбинированные (`multiplication`, `lower`, `digits`, `finalMult`, `finalPlus`, `plusOrMult`). В `Main.hs` добавлен квалифицированный импорт всех трёх парсеров.

- **Коммит 8. ParsecParser (23.04.2026, 20:03, fbea3dc)**

Реализован модуль `MyParsers.ParsecParser` на основе библиотеки `Parsec`: парсеры `digitP`, `multiplicationP`, `digitsP`, `finalMultP`, `finalPlusP`, `plusOrMultP` (с использованием `try` для `backtracking`). Функция `runParsecParser` запускает парсер и возвращает результат в виде `MyMaybe`.

- **Коммит 9. AttoparsecParser (23.04.2026, 21:19, 86715b9)**

Реализован модуль `MyParsers.AttoparsecParser` на основе библиотеки `Attoparsec`: аналогичные парсеры `digitA`, `multiplicationA`, `digitsA`, `finalMultA`, `finalPlusA`, `plusOrMultAttoparsec`. Функция `runParser` преобразует `String` в `Text` и возвращает результат в виде `MyMaybe`. Добавлен вывод результата для `Attoparsec`.

- **Коммит 10. Комментарии-промпты (24.04.2026, 11:40, 551327d)**

Добавлены комментарии-промпты, использованные при генерации модулей `Parsec` и `Attoparsec`.

5.4 Вопросы защиты и их решения

- **Вопрос 1 (задание дано 24.04.2026)**

Составить выражение с `<$>`, `<*>`, `<*>` типа `Either a (String, Float)`.

Решение:

```
1 (,) <$> (Right "abc") <*> (Right (+1.0) <*> (Right (4.0 :: Float)))
```

Листинг 3: `Either a (String, Float)` через аппликативный функтор

Первый `<*>` применяет функцию `(+1.0)` к значению `4.0`, второй — конструктор пары `(,)` к `"abc"` и полученному `5.0`. Результат: `Right ("abc 5.0")`.

- **Вопрос 2 (задание дано 08.05.2026)**

Составить выражение с `<$>`, `<*>`, `<*>` типа `Either a (Int, String)`.

Решение:

```
1 (,) <$> (Right (1 :: Integer)) <*> (Right (++ "1") <*> (Right ("abc" ::  
  ↪ String)))
```

Листинг 4: `Either a (Int, String)` через аппликативный функтор

Первый `<*>` применяет функцию `(++ "1")` к строке `"abc"`, второй — конструктор пары к `1` и `"abc1"`. Результат: `Right (1, "abc1")`.

6 Лабораторная работа №3

Третья работа состоит из двух частей: генерация глитч-эффектов на изображениях и поиск пути в лабиринте с использованием монады `RWS`.

6.1 Часть 1. Глитч-эффекты

- **Коммит 1. Создание проектов (06.05.2026, 17:36, 82f83ea)**

Добавлены файлы `stack`-проектов `myGlithes` и `myMaze`.

- **Коммит 2. Функции глитч-эффектов (07.05.2026, 13:17, 2489018)**

В модуле `Glitches` реализованы:

- a) `intToChar` — преобразование `Int` в `Char` по модулю 255;
- b) `intToBC` — преобразование `Int` в `ByteString` из одного символа;
- c) `replaceByte` — детерминированная замена байта в заданной позиции;
- d) `randomReplaceByte` — замена случайного байта на случайное значение;
- e) `sortSection` — сортировка участка `ByteString` по заданному смещению и размеру;
- f) `randomSortSection` — сортировка случайного участка фиксированного размера (25 байт).

В `Main.hs` имя файла берётся из аргументов командной строки, к изображению последовательно применяется цепочка глитч-функций (через `foldM`), каждая версия сохраняется в отдельный файл.

6.2 Часть 2. Лабиринт на монаде RWS

- Коммит 3. Типы данных лабиринта (07.05.2026, 18:50, 593ff49)

В модуле Maze определены типы: `RoomType = Start String | Finish String | Node String`, `Room = Room RoomType [String]` (тип комнаты и список соседей), `Maze = [Room]`. Реализованы функции `getName`, `getNeighbours`, `isFinish`, `findFinish`, `findStart`.

- Коммит 4. Поиск пути (08.05.2026, 00:24, 19cfd2b)

Добавлены функции `findRoom`, `isInPath`, `neighboursCount`, `getUnvisited`, а также основной решатель `solveMaze` на монаде RWS: окружение — лабиринт, состояние — пройденный путь, лог — журнал перемещений. Реализован вспомогательный обход `searchPath` (поиск в глубину) с логированием и обработкой ошибок (отсутствие старта/финиша, тупик). По требованию задания до-нотация не используется.

- Коммит 5. Тестирование и очистка (08.05.2026, 02:33, aed3fae)

В `Main.hs` добавлены тестовые лабиринты `schoolMaze` и `castleMaze` с выводом результата, пути и логов. Функции `findStart`/`findFinish` переработаны для устранения предупреждений компилятора; из исходников удалена кириллица.

6.3 Вопросы защиты и их решения

- Вопрос 1 (задание дано 08.05.2026)

Составить выражение вида `(_ >= _ >= _)` с монадой `State`.

Решение:

```
1 f1 = state (\x -> ("x", "log"))
2 f2 = (\x -> state (\x -> ("x", "log")))
3 runState (f1 >=> f2 >=> f2) "abc"
4 -- ("x", "log")
```

Листинг 5: Цепочка `>=` для монады `State`

7 Лабораторная работа №4

В четвёртой работе реализован собственный трансформер монад `MyStateT` и текстовая игра «прохождение лабиринта» на стеке из трёх трансформеров.

7.1 Коммиты и выполняемая работа

- **Коммит 1. Трансформер MyStateT (14.05.2026, 22:35, 374db69)**

Определён тип `newtype MyStateT s m a = MyStateT { runMyStateT :: s -> m (a, s) }`. Реализована функция `myState` (поднятие чистой функции преобразования состояния) и представители классов:

- a) `Functor` (при условии `Functor m`);
- b) `Applicative` (при условии `Monad m`);
- c) `Monad` (при условии `Monad m`);
- d) `MonadTrans` с методом `lift`.

- **Коммит 2. Загрузка лабиринта (14.05.2026, 23:39, a72da11)**

В модуль `Maze` перенесены типы и вспомогательные функции из третьей работы. Добавлены `parseRoom :: String -> Room` и `loadMaze :: FilePath -> IO Maze` для чтения карты лабиринта из текстового файла. Старт и финиш распознаются по именам «старт» и «финиш».

- **Коммит 3. Игровая логика (15.05.2026, 01:32, 947cbc8)**

Определён стек трансформеров `type Game a = ReaderT Maze (WriterT [String] (MyStateT String IO))` `a: ReaderT` — доступ к лабиринту, `WriterT` — логирование, `MyStateT` — текущая комната, `IO` — ввод-вывод. Реализованы функции `logPrint`, `getCRoom`, `setCRoom`, `runGame` (с явной терминацией трансформеров), основной игровой цикл `gameProc` и точка входа `playMaze`. Игрок перемещается по комнатам, может завершить игру вводом «q»; перемещения логируются и выводятся в конце.

- **Коммит 4. Тестовый лабиринт (15.05.2026, 01:33, 097370b)**

Добавлен файл `mazes/uni.maze` с тестовой картой лабиринта.

- **Коммит 5. Комментарии к runGame (15.05.2026, 17:56, 114b3fd)**

Добавлены подробные комментарии к функции `runGame`, поясняющие пошаговое развёртывание (терминацию) стека трансформеров.

7.2 Вопросы защиты и ответы на них

- **Вопрос 1**

Как работают типы в строке `runMyStateT (runWriterT (runReaderT game maze)) startRoom?`

Решение: применение функций снимает трансформеры по очереди:

- 1) `runReaderT game maze` даёт `WriterT [String] (MyStateT String IO)` `a`;
- 2) `runWriterT (...)` даёт `MyStateT String IO (a, [String])`;
- 3) `runMyStateT (...)` `startRoom` даёт `IO ((a, [String]), String)`.

• Вопрос 2

Что такое терминатор?

Решение: терминатор — это самая нижняя монада в стеке трансформеров, то есть базовая монада, поверх которой надстраиваются все остальные слои. В данном проекте терминатором является `IO`.

• Вопрос 3

Что такое монадный трансформер?

Решение: монадный трансформер — это тип вида `T m a`, параметризованный другой монадой `m`, который добавляет к `m` новый эффект, сохраняя возможность использовать эффекты `m`. Трансформер реализует класс `MonadTrans` с методом `lift :: m a -> T m a`, позволяющим «поднимать» вычисления внутренней монады в расширенный стек.

• Вопрос 4

Где запускается монада `IO`?

Решение: монада `IO` запускается через оператор `(<-)` в `do`-нотации.

8 Лабораторная работа №5

В пятой работе реализованы вспомогательные функции и набор свойств для их тестирования с помощью библиотеки `QuickCheck`.

8.1 Коммиты и выполняемая работа

• Коммит 1. Создание проекта (21.05.2026, 22:50, 4c889c4)

Создан `stack`-проект `myFunctionsAndTests`, в `package.yaml` добавлена зависимость `QuickCheck` для тестов.

• Коммит 2. Тестируемые функции (21.05.2026, 23:27, 37d2423)

В модуле `Lib` реализованы:

- a) `normMod :: Int -> Int -> Int` — приведение числа к остатку по модулю через повторное вычитание/сложение (вспомогательная функция);
- b) `addMod :: Int -> Int -> Int -> Int` — сложение по модулю;

- c) `splitTokens :: String -> [String]` — разбиение строки на чередующиеся блоки слов и пробелов;
- d) `clean :: [String] -> [String]` — удаление пустых токенов;
- e) `reverseWords :: String -> String` — изменение порядка слов на обратный (реализована как `concat . reverse . splitTokens`).

• Коммит 3. Генераторы и свойства (22.05.2026, 01:24, ccc2b26)

В `test/Spec.hs` реализованы генераторы `genMod`, `genWord`, `genSpaces`, `genPhrase`, `genText` и свойства:

- a) `prop_addMod_mod` — согласованность результата с `mod`: `addMod x y m mod == (x + y) mod`;
- b) `prop_addMod_neutral` — нейтральный элемент: `addMod x 0 m == x mod`;
- c) `prop_addMod_comm` — коммутативность: `addMod x y m == addMod y x m`;
- d) `prop_rev_empty` — переворот пустой строки даёт пустую строку;
- e) `prop_rev_single` — строка из одного слова не меняется;
- f) `prop_rev_order` — порядок слов меняется на обратный;
- g) `prop_rev_involution` — двойное применение возвращает исходную строку.

• Коммит 4. `verbose` и `withMaxSuccess` (22.05.2026, 01:29, c962812)

В `main` тесты обернуты в `verbose` и `withMaxSuccess 5` для подробного вывода ограниченного числа проверок.

• Коммит 5. `Dictionary` (22.05.2026, 17:56, f22c8331)

Добавлен словарь `dictionary :: [String]` для генерации предложений с осмысленными словами.

8.2 Задание преподавателя

Задание: необходимо сделать так, чтобы 2 из 3 тестов для сложения чисел по модулю не работали (завершались с ошибкой).

Решение: сохранить коммутативность (`addMod x y m == addMod y x m`), обеспечив неверный результат в остальных случаях. Для этого к сумме добавлена константа 20 перед приведением по модулю:

```

1 -- было:
2 addMod x y m = normMod (x + y) m
3 -- стало:
4 addMod x y m = normMod (x + y + 20) m

```

Листинг 6: `addMod` — намеренно неверная реализация

Прибавление 20 к сумме нарушает свойства согласованности с `mod` и нейтрального элемента, поскольку результат сдвигается на ненулевую константу. Коммутативность при этом сохраняется: $x + y + 20 == y + x + 20$. Таким образом, тесты `prop_addMod_mod` и `prop_addMod_neutral` завершаются с ошибкой, а `prop_addMod_comm` — проходит.

А Приложение А. Листинги кода

А.1 Лабораторная работа №1

practice_01/pr01_1.hs

```
1 -----
2 -- Практическое задание 1. Часть 1 --
3 -----
4
5 module Pr01_1 where
6
7 myZip :: [a] -> [b] -> [(a, b)]
8 myZip [] xs = []
9 myZip xs [] = []
10 myZip (x:xs) (y:ys) = (x,y) : myZip xs ys
11
12 myZip3 :: [a] -> [b] -> [c] -> [(a, b, c)]
13 myZip3 (x:xs) (y:ys) (z:zs) = (x, y, z) : myZip3 xs ys zs
14 myZip3 _ _ _ = []
15
16 myUnzip :: [(a, b)] -> ([a], [b])
17 myUnzip [] = ([], [])
18 myUnzip ((x, y) : xys) = (x : xs, y : ys)
19   where
20     (xs, ys) = myUnzip xys
21
22 myMap :: (a -> b) -> [a] -> [b]
23 myMap _ [] = []
24 myMap f (x : xs) = f x : myMap f xs
25
26 myZipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
27 myZipWith _ [] _ = []
28 myZipWith _ _ [] = []
29 myZipWith f (x:xs) (y:ys) = f x y : myZipWith f xs ys
30
31 myZipWith3 :: (a -> b -> c -> d) -> [a] -> [b] -> [c] -> [d]
32 myZipWith3 _ [] _ _ = []
33 myZipWith3 _ _ [] _ = []
34 myZipWith3 _ _ _ [] = []
35 myZipWith3 f (x:xs) (y:ys) (z:zs) = f x y z : myZipWith3 f xs ys zs
36
37 myFST :: (a, b, c) -> a
38 myFST (a, _, _) = a
39
40 mySND :: (a, b, c) -> b
41 mySND (_, a, _) = a
42
43 myTHRD :: (a, b, c) -> c
44 myTHRD (_, _, a) = a
45
46 myHead :: [a] -> a
47 myHead (x:xs) = x
48
49 myTail :: [a] -> [a]
50 myTail (x:xs) = xs
51
52 myTake :: Int -> [a] -> [a]
53 myTake 0 _ = []
```

```

54 myTake _ [] = []
55 myTake n (x:xs) = x : myTake (n - 1) xs
56
57 myDrop :: Int -> [a] -> [a]
58 myDrop 0 xs = xs
59 myDrop _ [] = []
60 myDrop n (x :xs) = myDrop (n - 1) xs
61
62 myProduct :: [Int] -> Int
63 myProduct [] = 1
64 myProduct (x:xs) = x * myProduct xs
65
66 myAll :: (a -> Bool) -> [a] -> Bool
67 myAll _ [] = True
68 myAll p (x:xs) = p x && myAll p xs
69
70 myAny :: (a -> Bool) -> [a] -> Bool
71 myAny _ [] = False
72 myAny p (x:xs) = p x || myAny p xs
73
74 myComposition :: (b -> c) -> (a -> b) -> a -> c
75 myComposition f g x = f (g x)

```

practice_01/pr01_2.hs

```

1 -----
2 -- Практические задание 1. Часть 2 --
3 -----
4
5 module Pr01_2 where
6
7 myFST :: (a, b, c) -> a
8 myFST (a, _, _) = a
9
10 mySND :: (a, b, c) -> b
11 mySND (_, a, _) = a
12
13 myTHRD :: (a, b, c) -> c
14 myTHRD (_, _, a) = a
15
16 myZipSave :: [a] -> [b] -> ([a, b], [a], [b])
17 myZipSave [] ys = ([], [], ys)
18 myZipSave xs [] = ([], xs, [])
19 myZipSave (x:xs) (y:ys) = ((x, y) : myFST res, mySND res, myTHRD res) where
20     res = myZipSave xs ys
21
22 myUnzipSave :: ([a, b], [a], [b]) -> ([a], [b])
23 myUnzipSave ([], restX, restY) = (restX, restY)
24 myUnzipSave ((x, y) : pairs, restX, restY) = (x : (fst res), y : (snd res)) where
25     res = myUnzipSave (pairs, restX, restY)
26
27 myFoldr :: (a -> b -> b) -> b -> [a] -> b
28 myFoldr f ini [] = ini
29 myFoldr f ini (x:xs) = x `f` myFoldr f ini xs
30
31 myFoldl :: (b -> a -> b) -> b -> [a] -> b
32 myFoldl f ini [] = ini
33 myFoldl f ini (x:xs) = myFoldl f (f ini x) xs

```

```

34
35 myReverse :: [a] -> [a]
36 myReverse = myFoldl (flip (:)) []
37
38 myFoldl1 :: (a -> a -> a) -> [a] -> a
39 myFoldl1 f (x:xs) = myFoldl f x xs
40 myFoldl1 _ [] = error "empty list"
41
42 myFoldr1 :: (a -> a -> a) -> [a] -> a
43 myFoldr1 _ [x] = x
44 myFoldr1 f (x:xs) = f x (myFoldr1 f xs)
45 myFoldr1 _ [] = error "empty list"
46
47 myTakeWhile :: (a -> Bool) -> [a] -> [a]
48 myTakeWhile _ [] = []
49 myTakeWhile pred (x:xs) = myFoldr f [] (x:xs) where
50     f x lst = if pred x == True then x : lst else lst
51
52 mySpan :: (a -> Bool) -> [a] -> ([a], [a])
53 mySpan _ [] = ([], [])
54 mySpan pred (x:xs) = myFoldr f ([], []) (x:xs) where
55     f x (sat, unsat) | pred x && null unsat = (x : sat, unsat)
56                     | otherwise           = (sat, x : unsat)
57
58 myMaybe :: b -> (a -> b) -> Maybe a -> b
59 myMaybe deflt _ Nothing = deflt
60 myMaybe _ f (Just x) = f x
61
62 myUnfoldr :: (b -> Maybe (a, b)) -> b -> [a]
63 myUnfoldr f ini = helper (f ini) where
64     helper (Just (x,ini')) = x : myUnfoldr f ini'
65     helper Nothing         = []
66
67 data MyList a = MyEmpty | MyCons a (MyList a) deriving Show
68
69 myMap :: (a -> b) -> MyList a -> MyList b
70 myMap _ MyEmpty      = MyEmpty
71 myMap f (MyCons x xs) = MyCons (f x) (myMap f xs)
72
73 data Ingredient = Oil Int | Chocolate Int | Vanilla Int | Carrot Int
74                | Egg Int | Flour Int | Shugar Int | BakingPowder Int
75                deriving Show
76
77 data FillingMix = NotMix | SpoiledMix | OilChocolateMix | OilVanillaMix
78                | OilCarrotMix deriving Show
79
80 data Dough = NotDough | SpoiledDough | CakeDough deriving Show
81
82 data CakeDoughType = NotCakeDough | SpoiledCakeDough | ChocolateCakeDough
83                    | VanillaCakeDough | CarrotCakeDough deriving Show
84
85 data Cake = NotCake | SpoiledCake | ChocolateCake | VanillaCake | CarrotCake
86            deriving Show
87
88 data Action = Bake deriving Show
89
90 makeCakeMix :: Ingredient -> Ingredient -> FillingMix
91 makeCakeMix (Oil x) (Chocolate y) | x >= 200 && y >= 20 = OilChocolateMix
92 makeCakeMix (Chocolate x) (Oil y) | x >= 20 && y >= 200 = OilChocolateMix
93 makeCakeMix (Oil x) (Vanilla y) | x >= 200 && y >= 15 = OilVanillaMix
94 makeCakeMix (Vanilla x) (Oil y) | x >= 15 && y >= 200 = OilVanillaMix
95 makeCakeMix (Oil x) (Carrot y) | x >= 200 && y >= 50 = OilCarrotMix

```

```

96 makeCakeMix (Carrot x) (Oil y)      | x >= 50 && y >= 200 = OilCarrotMix
97 makeCakeMix (Oil x) (Oil y)        | x >= 200 && y >= 200 = SpoiledMix
98 makeCakeMix _ _                    = NotMix
99
100 cakeDough :: Ingredient -> Ingredient -> Ingredient -> Ingredient -> Dough
101 cakeDough (Egg e) (Flour f) (Sugar s) (BakingPowder b)
102     | e >= 8 && f >= 200 && s >= 100 && b >= 2 = CakeDough
103 cakeDough (Oil _) _ _ _ = SpoiledDough
104 cakeDough _ _ _ _ = NotDough
105
106 chocolateCakeDough :: Dough -> FillingMix -> CakeDoughType
107 chocolateCakeDough SpoiledDough _ = SpoiledCakeDough
108 chocolateCakeDough _ SpoiledMix = SpoiledCakeDough
109 chocolateCakeDough CakeDough OilChocolateMix = ChocolateCakeDough
110 chocolateCakeDough _ _ = NotCakeDough
111
112 vanillaCakeDough :: Dough -> FillingMix -> CakeDoughType
113 vanillaCakeDough SpoiledDough _ = SpoiledCakeDough
114 vanillaCakeDough _ SpoiledMix = SpoiledCakeDough
115 vanillaCakeDough CakeDough OilVanillaMix = VanillaCakeDough
116 vanillaCakeDough _ _ = NotCakeDough
117
118 carrotCakeDough :: Dough -> FillingMix -> CakeDoughType
119 carrotCakeDough SpoiledDough _ = SpoiledCakeDough
120 carrotCakeDough _ SpoiledMix = SpoiledCakeDough
121 carrotCakeDough CakeDough OilCarrotMix = CarrotCakeDough
122 carrotCakeDough _ _ = NotCakeDough
123
124 chocolateCake :: CakeDoughType -> Action -> Cake
125 chocolateCake SpoiledCakeDough _ = SpoiledCake
126 chocolateCake ChocolateCakeDough Bake = ChocolateCake
127 chocolateCake _ _ = NotCake
128
129 vanillaCake :: CakeDoughType -> Action -> Cake
130 vanillaCake SpoiledCakeDough _ = SpoiledCake
131 vanillaCake VanillaCakeDough Bake = VanillaCake
132 vanillaCake _ _ = NotCake
133
134 carrotCake :: CakeDoughType -> Action -> Cake
135 carrotCake SpoiledCakeDough _ = SpoiledCake
136 carrotCake CarrotCakeDough Bake = CarrotCake
137 carrotCake _ _ = NotCake

```

A.2 Лабораторная работа №2

practice_02/part1/MyEvolProject/app/MyEvolModule.hs

```

1 module MyEvolModule where
2
3 data MyEvolution = LUCA | Cyanobacteria | Trilobite | Ichthyostega
4                  | Dimetrodon | Archaeopteryx | Morganucodon | Purgatorius
5                  | Australopithecine | Humans
6
7 instance Show MyEvolution where
8     show LUCA          = "Last Universal Common Ancestor"
9     show Cyanobacteria = "Synechococcus"
10    show Trilobite      = "Paradoxides"

```

```

11     show Ichthyostega      = "Ichthyostega"
12     show Dimetrodon       = "Dimetrodon"
13     show Archaeopteryx    = "Archaeopteryx"
14     show Morganucodon     = "Morganucodon"
15     show Purgatorius      = "Purgatorius"
16     show Australopithecine = "Australopithecus Afarensis"
17     show Humans           = "Homo Sapiens"
18
19 instance Read MyEvolution where
20     readsPrec _ "Last Universal Common Ancestor" = [(LUCA, "")]
21     readsPrec _ "Synechococcus"                  = [(Cyanobacteria, "")]
22     readsPrec _ "Paradoxides"                    = [(Trilobite, "")]
23     readsPrec _ "Ichthyostega"                    = [(Ichthyostega, "")]
24     readsPrec _ "Dimetrodon"                      = [(Dimetrodon, "")]
25     readsPrec _ "Archaeopteryx"                   = [(Archaeopteryx, "")]
26     readsPrec _ "Morganucodon"                    = [(Morganucodon, "")]
27     readsPrec _ "Purgatorius"                     = [(Purgatorius, "")]
28     readsPrec _ "Australopithecus Afarensis"      = [(Australopithecine, "")]
29     readsPrec _ "Homo Sapiens"                    = [(Humans, "")]
30     readsPrec _ _                                 = []
31
32 instance Eq MyEvolution where
33     LUCA == LUCA = True
34     Cyanobacteria == Cyanobacteria = True
35     Trilobite == Trilobite = True
36     Ichthyostega == Ichthyostega = True
37     Dimetrodon == Dimetrodon = True
38     Archaeopteryx == Archaeopteryx = True
39     Morganucodon == Morganucodon = True
40     Purgatorius == Purgatorius = True
41     Australopithecine == Australopithecine = True
42     Humans == Humans = True
43     _ == _ = False
44
45 instance Enum MyEvolution where
46     fromEnum LUCA          = 0
47     fromEnum Cyanobacteria  = 1
48     fromEnum Trilobite     = 2
49     fromEnum Ichthyostega   = 3
50     fromEnum Dimetrodon    = 4
51     fromEnum Archaeopteryx = 5
52     fromEnum Morganucodon  = 6
53     fromEnum Purgatorius   = 7
54     fromEnum Australopithecine = 8
55     fromEnum Humans        = 9
56     toEnum 0 = LUCA
57     toEnum 1 = Cyanobacteria
58     toEnum 2 = Trilobite
59     toEnum 3 = Ichthyostega
60     toEnum 4 = Dimetrodon
61     toEnum 5 = Archaeopteryx
62     toEnum 6 = Morganucodon
63     toEnum 7 = Purgatorius
64     toEnum 8 = Australopithecine
65     toEnum 9 = Humans
66     toEnum _ = error "Invalid MyEvolution value"
67
68 instance Ord MyEvolution where
69     compare x y = compare (fromEnum x) (fromEnum y)
70
71 instance Bounded MyEvolution where
72     minBound = LUCA

```

```

73     maxBound = Humans
74
75 data MyEvolution' = LUCA' | Cyanobacteria' | Trilobite' | Ichthyostega'
76                   | Dimetrodon' | Archaeopteryx' | Morganucodon' | Purgatorius'
77                   | Australopithecine' | Humans'
78     deriving (Show, Read, Eq, Ord, Enum, Bounded)

```

practice_02/part1/MyEvolProject/app/Main.hs

```

1 module Main where
2
3 import MyEvolModule
4 import Data.List (sort)
5
6 main :: IO ()
7 main = putStrLn $ show $ sort ([LUCA .. Humans] :: [MyEvolution])

```

practice_02/part2/myParser/src/MyTypes/MyMaybe.hs

```

1 module MyTypes.MyMaybe ( MyMaybe(MyJust, MyNothing) ) where
2
3 data MyMaybe a = MyJust a | MyNothing deriving (Show, Eq, Read)
4
5 instance Functor MyMaybe where
6     fmap _ MyNothing = MyNothing
7     fmap f (MyJust a) = MyJust (f a)
8
9 instance Applicative MyMaybe where
10     pure = MyJust
11     MyJust f <*> m = fmap f m
12     MyNothing <*> _ = MyNothing
13
14 instance Semigroup a => Semigroup (MyMaybe a) where
15     MyNothing <> b = b
16     a <> MyNothing = a
17     MyJust a <> MyJust b = MyJust (a <> b)
18
19 instance Semigroup a => Monoid (MyMaybe a) where
20     mempty = MyNothing
21
22 instance Foldable MyMaybe where
23     foldr _ z MyNothing = z
24     foldr f z (MyJust x) = f x z
25     foldMap _ MyNothing = mempty
26     foldMap f (MyJust x) = f x

```

practice_02/part2/myParser/src/MyTypes/MyTree.hs

```

1 module MyTypes.MyTree ( MyTree(Leaf, Node) ) where
2
3 data MyTree a = Leaf a | Node a (MyTree a) (MyTree a) deriving (Show, Eq, Read)
4

```

```

5 instance Functor MyTree where
6     fmap f (Leaf x)           = Leaf (f x)
7     fmap f (Node x left right) = Node (f x) (fmap f left) (fmap f right)
8
9 instance Foldable MyTree where
10    foldr f z (Leaf x)           = f x z
11    foldr f z (Node x left right) = f x (foldr f (foldr f z right) left)
12    foldMap f (Leaf x)           = f x
13    foldMap f (Node x left right) = f x <> foldMap f left <> foldMap f right
14    length (Leaf _)              = 1
15    length (Node _ l r)          = length l + length r + 1
16
17 instance Applicative MyTree where
18    pure = Leaf
19    (Leaf g) <*> x = fmap g x
20    (Node f leftF rightF) <*> (Leaf x) =
21        Node (f x) (leftF <*> Leaf x) (rightF <*> Leaf x)
22    (Node f leftF rightF) <*> (Node x left right) =
23        Node (f x) (leftF <*> left) (rightF <*> right)

```

practice_02/part2/myParser/src/MyTypes/MyEither.hs

```

1 module MyTypes.MyEither ( MyEither(MyLeft, MyRight) ) where
2
3 data MyEither e a = MyLeft e | MyRight a deriving (Show, Eq, Read)
4
5 instance Functor (MyEither e) where
6     fmap _ (MyLeft x)  = MyLeft x
7     fmap f (MyRight x) = MyRight (f x)
8
9 instance Foldable (MyEither e) where
10    foldr _ z (MyLeft _)  = z
11    foldr f z (MyRight x) = f x z
12    foldMap _ (MyLeft _)  = mempty
13    foldMap f (MyRight x) = f x
14
15 instance Applicative (MyEither e) where
16    pure = MyRight
17    MyLeft e <*> _          = MyLeft e
18    _ <*> MyLeft e          = MyLeft e
19    MyRight f <*> MyRight x = MyRight (f x)
20
21 instance Semigroup a => Semigroup (MyEither e a) where
22    MyLeft x <> _          = MyLeft x
23    _ <> MyLeft y          = MyLeft y
24    MyRight x <> MyRight y = MyRight (x <> y)

```

practice_02/part2/myParser/src/MyParsers/MyParser.hs

```

1 module MyParsers.MyParser (
2     Parser, runParser, satisfy, char, lower, digit, multiplication,
3     lowers, digits, finalMult, finalPlus, plusOrMult, optional
4 ) where
5
6 import MyTypes.MyMaybe

```

```

7 import Control.Applicative (Alternative(..))
8 import Data.Char
9
10 newtype Parser tok a = Parser { runParser :: [tok] -> MyMaybe ([tok], a) }
11
12 instance Functor (Parser tok) where
13     fmap g (Parser u) = Parser f where
14         f xs = case u xs of
15             MyNothing      -> MyNothing
16             MyJust (toks', x) -> MyJust (toks', g x)
17
18 instance Applicative (Parser tok) where
19     pure x = Parser $ \xs -> MyJust (xs, x)
20     Parser u <*> Parser v = Parser f where
21         f xs = case u xs of
22             MyNothing      -> MyNothing
23             MyJust (tok, x) -> case v tok of
24                 MyNothing      -> MyNothing
25                 MyJust (tok', x') -> MyJust (tok', x x')
26
27 instance Alternative (Parser tok) where
28     empty = Parser $ \_ -> MyNothing
29     Parser u <|> Parser v = Parser f where
30         f xs = case u xs of
31             MyNothing -> v xs
32             x         -> x
33
34 optional :: Alternative f => f a -> f (Maybe a)
35 optional v = Just <$> v <|> pure Nothing
36
37 satisfy :: (Char -> Bool) -> Parser Char Char
38 satisfy c = Parser f where
39     f (x:xs) | c x = MyJust (xs, x)
40     f _           = MyNothing
41
42 char :: Char -> Parser Char Char
43 char c = satisfy (== c)
44
45 lower :: Parser Char Char
46 lower = satisfy isLower
47
48 digit :: Parser Char Int
49 digit = digitToInt <$> satisfy isDigit
50
51 multiplication :: Parser Char Int
52 multiplication = (*) <$> digit <*> char '*' <*> digit
53
54 lowers :: Parser Char String
55 lowers = (:) <$> lower <*> lowers <|> pure ""
56
57 digits :: Parser Char Int
58 digits = foldl (\x y -> x * 10 + y) 0 <$> some digit
59
60 finalMult :: Parser Char Int
61 finalMult = (*) <$> digits <*> char '*' <*> digits
62
63 finalPlus :: Parser Char Int
64 finalPlus = (+) <$> digits <*> char '+' <*> digits
65
66 plusOrMult :: Parser Char Int
67 plusOrMult = finalMult <|> finalPlus

```

practice_02/part2/myParser/src/MyParsers/ParsecParser.hs

```
1 module MyParsers.ParsecParser (
2     Parser, runParsecParser, digitP, multiplicationP, digitsP,
3     finalMultP, finalPlusP, plusOrMultParsec
4 ) where
5
6 import Text.Parsec.String (Parser)
7 import Text.Parsec (digit, char, try, (<|>), parse)
8 import Data.Char (digitToInt)
9 import MyTypes.MyMaybe (MyMaybe(MyJust, MyNothing))
10
11 digitP :: Parser Int
12 digitP = digitToInt <$> digit
13
14 multiplicationP :: Parser Int
15 multiplicationP = (*) <$> digitP <*> char '*' <*> digitP
16
17 digitsP :: Parser Int
18 digitsP = foldl (\acc d -> acc * 10 + d) 0 <$> many1 digitP
19     where
20         many1 p = (:) <$> p <*> many p
21         many p = many1 p <|> pure []
22
23 finalMultP :: Parser Int
24 finalMultP = (*) <$> digitsP <*> char '*' <*> digitsP
25
26 finalPlusP :: Parser Int
27 finalPlusP = (+) <$> digitsP <*> char '+' <*> digitsP
28
29 plusOrMultParsec :: Parser Int
30 plusOrMultParsec = try finalMultP <|> finalPlusP
31
32 runParsecParser :: Parser Int -> String -> MyMaybe (String, Int)
33 runParsecParser p input =
34     case parse p "" input of
35         Left _      -> MyNothing
36         Right result -> MyJust ("", result)
```

practice_02/part2/myParser/src/MyParsers/AttoparsecParser.hs

```
1 module MyParsers.AttoparsecParser (
2     Parser, runParser, digitA, multiplicationA, digitsA,
3     finalMultA, finalPlusA, plusOrMultAttoparsec
4 ) where
5
6 import Data.Attoparsec.Text
7 import Control.Applicative (some, (<|>))
8 import MyTypes.MyMaybe (MyMaybe(..))
9 import Data.Char (digitToInt)
10 import qualified Data.Text as T
11
12 digitA :: Parser Int
13 digitA = digitToInt <$> digit
14
15 multiplicationA :: Parser Int
16 multiplicationA = (*) <$> digitA <*> char '*' <*> digitA
```

```

17
18 digitsA :: Parser Int
19 digitsA = Prelude.foldl (\x y -> x * 10 + y) 0 <$> some digitA
20
21 finalMultA :: Parser Int
22 finalMultA = (*) <$> digitsA <* char '*' <*> digitsA
23
24 finalPlusA :: Parser Int
25 finalPlusA = (+) <$> digitsA <* char '+' <*> digitsA
26
27 plusOrMultAttoparsec :: Parser Int
28 plusOrMultAttoparsec = try finalMultA <|> finalPlusA
29
30 runParser :: Parser Int -> String -> MyMaybe (String, Int)
31 runParser p input = case parseOnly ((,) <$> p <*> takeText) (T.pack input) of
32     Left _         -> MyNothing
33     Right (result, rest) -> MyJust (T.unpack rest, result)

```

practice_02/part2/myParser/app/Main.hs

```

1 module Main (main) where
2
3 import MyTypes.MyTree()
4 import MyTypes.MyMaybe()
5 import MyTypes.MyEither()
6
7 import qualified MyParsers.MyParser      as My
8 import qualified MyParsers.ParsecParser  as Parsec
9 import qualified MyParsers.AttoparsecParser as Atto
10
11 main :: IO ()
12 main = do
13     putStrLn "\nMyParser:"
14     putStrLn $ show (My.runParser My.plusOrMult "12*345dsf")
15     putStrLn $ show (My.runParser My.plusOrMult "12+345dsf")
16     putStrLn "Parsec:"
17     putStrLn $ show (Parsec.runParsecParser Parsec.plusOrMultParsec "12*345dsf")
18     putStrLn $ show (Parsec.runParsecParser Parsec.plusOrMultParsec "12+345dsf")
19     putStrLn "Attoparsec:"
20     putStrLn $ show (Atto.runParser Atto.plusOrMultAttoparsec "12*345dsf")
21     putStrLn $ show (Atto.runParser Atto.plusOrMultAttoparsec "12+345dsf")

```

A.3 Лабораторная работа №3

practice_03/myGlithes/src/Glitches.hs

```

1 module Glitches where
2
3 import System.Random
4 import qualified Data.ByteString.Char8 as BC
5
6 intToChar :: Int -> Char
7 intToChar int = toEnum safeInt
8     where safeInt = int `mod` 255

```

```

9
10 intToBC :: Int -> BC.ByteString
11 intToBC int = BC.pack [intToChar int]
12
13 replaceByte :: Int -> Int -> BC.ByteString -> BC.ByteString
14 replaceByte loc chV bytes = mconcat [before, newChar, after]
15   where (before, rest) = BC.splitAt loc bytes
16         after          = BC.drop 1 rest
17         newChar         = intToBC chV
18
19 randomReplaceByte :: BC.ByteString -> IO BC.ByteString
20 randomReplaceByte bytes = do
21   let bytesLength = BC.length bytes
22   location <- randomRIO (1, bytesLength)
23   chV      <- randomRIO (0, 255)
24   return (replaceByte location chV bytes)
25
26 sortSection :: Int -> Int -> BC.ByteString -> BC.ByteString
27 sortSection start size bytes = mconcat [before, changed, after]
28   where (before, rest) = BC.splitAt start bytes
29         (target, after) = BC.splitAt size rest
30         changed         = BC.reverse (BC.sort target)
31
32 randomSortSection :: BC.ByteString -> IO BC.ByteString
33 randomSortSection bytes = do
34   let sectionSize = 25
35   let bytesLength = BC.length bytes
36   start <- randomRIO (0, bytesLength - sectionSize)
37   return (sortSection start sectionSize bytes)

```

practice_03/myGlithes/app/Main.hs

```

1 module Main (main) where
2
3 import System.Environment
4 import qualified Data.ByteString.Char8 as BC
5 import Control.Monad (foldM)
6 import Glitches (randomReplaceByte, randomSortSection)
7
8 main :: IO ()
9 main = do
10   args <- getArgs
11   let fileName = head args
12   imageFile <- BC.readFile fileName
13
14   glitched1 <- randomReplaceByte imageFile
15   let glitchedFileName1 = mconcat ["glitched_v1_", fileName]
16   BC.writeFile glitchedFileName1 glitched1
17   putStrLn $ "ToToBo! " ++ glitchedFileName1
18
19   glitched2 <- randomSortSection imageFile
20   let glitchedFileName2 = mconcat ["glitched_v2_", fileName]
21   BC.writeFile glitchedFileName2 glitched2
22   putStrLn $ "ToToBo! " ++ glitchedFileName2
23
24   glitched3 <- foldM (\bytes func -> func bytes) imageFile
25                   [ randomReplaceByte, randomSortSection
26                     , randomReplaceByte, randomSortSection

```

```

27         , randomReplaceByte ]
28     let glitchedFileName3 = mconcat ["glitched_v3_", fileName]
29     BC.writeFile glitchedFileName3 glitched3
30     putStrLn $ "ToToBo! " ++ glitchedFileName3

```

practice_03/myMaze/src/Maze.hs

```

1 module Maze (
2     RoomType(..), Room(..), Maze,
3     solveMaze, getName, getNeighbours,
4     findRoom, findStart, findFinish,
5 ) where
6
7 import qualified Control.Monad.RWS as RWS
8
9 data RoomType = Start String | Finish String | Node String deriving (Eq)
10 data Room     = Room RoomType [String] deriving (Eq)
11 type Maze     = [Room]
12
13 getName :: Room -> String
14 getName (Room (Start name) _) = name
15 getName (Room (Finish name) _) = name
16 getName (Room (Node name)   _) = name
17
18 getNeighbours :: Room -> [String]
19 getNeighbours (Room _ x) = x
20
21 isFinish :: Room -> Bool
22 isFinish (Room (Finish _) _) = True
23 isFinish _                    = False
24
25 findFinish :: Maze -> Maybe Room
26 findFinish [] = Nothing
27 findFinish ((Room (Finish name) ns) : _) = Just (Room (Finish name) ns)
28 findFinish ((Room (Start _) _) : rest)   = findFinish rest
29 findFinish ((Room (Node _) _) : rest)    = findFinish rest
30
31 findStart :: Maze -> Maybe Room
32 findStart [] = Nothing
33 findStart ((Room (Start name) ns) : _) = Just (Room (Start name) ns)
34 findStart ((Room (Finish _) _) : rest) = findStart rest
35 findStart ((Room (Node _) _) : rest)  = findStart rest
36
37 findRoom :: Maze -> String -> Maybe Room
38 findRoom [] _ = Nothing
39 findRoom (room : rest) seek
40     | getName room == seek = Just room
41     | otherwise           = findRoom rest seek
42
43 isInPath :: [String] -> String -> Bool
44 isInPath [] _ = False
45 isInPath (n:rest) seek | n == seek = True
46                       | otherwise = isInPath rest seek
47
48 getUnvisited :: [String] -> [String] -> [String]
49 getUnvisited [] _ = []
50 getUnvisited (n:ns) path | isInPath path n = getUnvisited ns path
51                       | otherwise         = n : getUnvisited ns path

```

```

52
53 solveMaze :: RWS.RWS Maze [String] [String] (Maybe [String])
54 solveMaze =
55     RWS.ask >>= \maze ->
56     case (findStart maze, findFinish maze) of
57         (Nothing, _) ->
58             RWS.tell ["Error: Start room not found"] >> return Nothing
59         (_, Nothing) ->
60             RWS.tell ["Error: Finish room not found"] >> return Nothing
61         (Just startRoom, Just finishRoom) ->
62             let startName = getName startRoom
63                 finishName = getName finishRoom
64             in RWS.put [startName] >>
65                 RWS.tell ["Start at room: " ++ startName] >>
66                 RWS.tell ["Finish at room: " ++ finishName] >>
67                 searchPath maze finishName
68
69 searchPath :: Maze -> String -> RWS.RWS Maze [String] [String] (Maybe [String])
70 searchPath maze finishName =
71     RWS.get >>= \path ->
72     case path of
73         [] -> return Nothing
74         (currentName : _) ->
75             case findRoom maze currentName of
76                 Nothing ->
77                     RWS.tell ["Error: Room " ++ currentName ++ " not found"] >>
78                     return Nothing
79                 Just currentRoom ->
80                     if isFinish currentRoom
81                         then RWS.tell ["Finish: Reached room " ++ finishName] >>
82                             return (Just (reverse path))
83                     else let unvisited = getUnvisited (getNeighbours currentRoom) path
84                         in case unvisited of
85                             [] ->
86                                 RWS.tell ["Dead end: " ++ currentName] >>
87                                 RWS.tell ["Failure: Cannot reach finish"] >>
88                                 return Nothing
89                             (nextRoom : _) ->
90                                 RWS.tell ["Move: " ++ currentName ++ " -> " ++ nextRoom]
91                                 ↵ >>
92                                 RWS.put (nextRoom : path) >>
93                                 searchPath maze finishName

```

practice_03/myMaze/app/Main.hs

```

1 module Main where
2
3 import qualified Control.Monad.RWS as RWS
4 import Maze
5
6 main :: IO ()
7 main = do
8     putStrLn "\n School Maze"
9     let (result, path, logs) = RWS.runRWS solveMaze schoolMaze []
10    putStrLn $ "Result: " ++ show result
11    putStrLn $ "Path: " ++ show path
12    putStrLn "Log:"
13    mapM_ putStrLn logs

```

```

14
15     putStrLn "\n Cemetery Maze"
16     let (result2, path2, logs2) = RWS.runRWS solveMaze cemeteryMaze []
17     putStrLn $ "Result: " ++ show result2
18     putStrLn $ "Path: " ++ show path2
19     putStrLn "Log:"
20     mapM_ putStrLn logs2
21
22 schoolMaze :: Maze
23 schoolMaze =
24     [ Room (Start "Entrance")      ["Hall"]
25     , Room (Node "Hall")           ["Entrance", "Mathematics", "Physical Education"]
26     , Room (Node "Mathematics")    ["Hall", "Computer Science"]
27     , Room (Node "Computer Science") ["Mathematics", "Principal's Office"]
28     , Room (Finish "Principal's Office") ["Computer Science"]
29     , Room (Node "Physical Education") ["Hall"]
30     ]
31
32 cemeteryMaze :: Maze
33 cemeteryMaze =
34     [ Room (Start "Gate")          ["Alley"]
35     , Room (Node "Alley")          ["Gate", "Chapel", "Crypt"]
36     , Room (Node "Chapel")        ["Alley", "Tomb"]
37     , Room (Finish "Tomb")        ["Chapel"]
38     , Room (Node "Crypt")         ["Alley"]
39     ]

```

A.4 Лабораторная работа №4

practice_04/myMazeGame/src/MyStateT.hs

```

1 {-# LANGUAGE InstanceSigs #-}
2
3 module MyStateT (MyStateT(..), myState, lift) where
4
5 import Control.Monad.Trans.Class (MonadTrans(..))
6
7 newtype MyStateT s m a = MyStateT { runMyStateT :: s -> m (a, s) }
8
9 myState :: Monad m => (s -> (a, s)) -> MyStateT s m a
10 myState f = MyStateT $ \s -> return . f
11
12 instance Functor m => Functor (MyStateT s m) where
13     fmap :: (a -> b) -> MyStateT s m a -> MyStateT s m b
14     fmap f x = MyStateT $ \s -> fmap (\(a, sx) -> (f a, sx)) (runMyStateT x s)
15
16 instance Monad m => Applicative (MyStateT s m) where
17     pure :: a -> MyStateT s m a
18     pure x = MyStateT $ \s -> pure (x, s)
19     (<*>) :: MyStateT s m (a -> b) -> MyStateT s m a -> MyStateT s m b
20     f <*> v = MyStateT $ \s -> do
21         (fx, s1) <- runMyStateT f s
22         (a, s2) <- runMyStateT v s1
23         return (fx a, s2)
24
25 instance Monad m => Monad (MyStateT s m) where
26     (>>=) :: MyStateT s m a -> (a -> MyStateT s m b) -> MyStateT s m b

```

```

27     m >>= k = MyStateT $ \s -> do
28         (a, sx) <- runMyStateT m s
29         runMyStateT (k a) sx
30
31 instance MonadTrans (MyStateT s) where
32     lift :: Monad m => m a -> MyStateT s m a
33     lift m = MyStateT $ \s -> do
34         a <- m
35         return (a, s)

```

practice_04/myMazeGame/src/Maze.hs

```

1 module Maze (RoomType(..), Room(..), Maze, getName, getNeighbours, isFinish,
2             findRoom, findStart, isInPath, loadMaze, playMaze) where
3
4 import Control.Monad.Reader as R
5 import Control.Monad.Writer as W
6 import MyStateT as M
7
8 data RoomType = Start String | Finish String | Node String deriving (Eq)
9 data Room     = Room RoomType [String] deriving (Eq)
10 type Maze     = [Room]
11
12 getName :: Room -> String
13 getName (Room (Start name) _) = name
14 getName (Room (Finish name) _) = name
15 getName (Room (Node name)   _) = name
16
17 getNeighbours :: Room -> [String]
18 getNeighbours (Room _ x) = x
19
20 isFinish :: Room -> Bool
21 isFinish (Room (Finish _) _) = True
22 isFinish _                   = False
23
24 findStart :: Maze -> Maybe Room
25 findStart [] = Nothing
26 findStart ((Room (Start name) ns) : _) = Just (Room (Start name) ns)
27 findStart (_ : rest) = findStart rest
28
29 findRoom :: Maze -> String -> Maybe Room
30 findRoom [] _ = Nothing
31 findRoom (room : rest) seek
32     | getName room == seek = Just room
33     | otherwise           = findRoom rest seek
34
35 isInPath :: [String] -> String -> Bool
36 isInPath [] _ = False
37 isInPath (n:rest) seek | n == seek = True
38                       | otherwise = isInPath rest seek
39
40 parseRoom :: String -> Room
41 parseRoom line =
42     let (name : neighbours) = words line
43     in roomType = case name of
44         "старт" -> Start name
45         "финиш" -> Finish name
46         _       -> Node name

```

```

47     in Room roomType neighbours
48
49 loadMaze :: FilePath -> IO Maze
50 loadMaze path = map parseRoom . filter (not . null) . lines <$> readFile path
51
52 type Game a = ReaderT Maze (WriterT [String] (MyStateT String IO)) a
53
54 io :: IO a -> Game a
55 io = R.lift . W.lift . M.lift
56
57 logPrint :: String -> Game ()
58 logPrint msg = R.lift (tell [msg]) >> io (putStrLn msg)
59
60 getCRoom :: Game String
61 getCRoom = R.lift $ W.lift $ myState $ \s -> (s, s)
62
63 setCRoom :: String -> Game ()
64 setCRoom name = R.lift $ W.lift $ myState $ \_ -> ((), name)
65
66 runGame :: Maze -> String -> Game a -> IO (a, [String], String)
67 runGame maze startRoom game = do
68     (result, fin) <- runMyStateT (runWriterT (runReaderT game maze)) startRoom
69     let (a, log) = result
70     return (a, log, fin)
71
72 gameProc :: Game ()
73 gameProc = do
74     maze <- ask
75     current <- getCRoom
76     case findRoom maze current of
77         Nothing -> logPrint "Room not found!"
78         Just room ->
79             if isFinish room
80                 then logPrint $ "You reached " ++ current ++ "!"
81                 else do
82                     logPrint $ "\nRoom: " ++ current
83                     logPrint $ "Can go: " ++ unwords (getNeighbours room)
84                     io $ putStr "Next room ('q' to quit): "
85                     input <- io getLine
86                     if input == "q"
87                         then logPrint "Quit."
88                         else if isInPath (getNeighbours room) input
89                             then setCRoom input >> gameProc
90                             else logPrint ("No path to " ++ input) >> gameProc
91
92 playMaze :: FilePath -> IO ()
93 playMaze path = do
94     maze <- loadMaze path
95     case findStart maze of
96         Nothing -> putStrLn "No start!"
97         Just start -> do
98             let startName = getName start
99             (_, logs, _) <- runGame maze startName gameProc
100             putStrLn "\nLog:"
101             mapM_ putStrLn logs

```

practice_04/myMazeGame/app/Main.hs

```

1 module Main (main) where

```

```

2
3 import Maze (playMaze)
4
5 main :: IO ()
6 main = playMaze "mazes/uni.maze"

```

A.5 Лабораторная работа №5

practice_05/myFunctionsAndTests/src/Lib.hs

```

1 module Lib
2   ( addMod
3   , reverseWords
4   , splitTokens
5   , clean
6   ) where
7
8 normMod :: Int -> Int -> Int
9 normMod x m
10  | x < 0      = normMod (x + m) m
11  | x < m      = x
12  | otherwise = normMod (x - m) m
13
14 addMod :: Int -> Int -> Int -> Int
15 addMod x y m = normMod (x + y + 20) m
16
17 splitTokens :: String -> [String]
18 splitTokens [] = []
19 splitTokens (c:cs) =
20   let isSpace      = c == ' '
21       (token, rest) = span (\x -> (x == ' ') == isSpace) (c:cs)
22   in token : splitTokens rest
23
24 clean :: [String] -> [String]
25 clean = filter (not . null)
26
27 reverseWords :: String -> String
28 reverseWords = concat . reverse . splitTokens

```

practice_05/myFunctionsAndTests/test/Spec.hs

```

1 import Test.QuickCheck
2 import Lib
3
4 genMod :: Gen Int
5 genMod = choose (1, 1000)
6
7 dictionary :: [String]
8 dictionary =
9   [ "cat", "dog", "house", "tree", "river", "mountain", "code", "haskell"
10   , "function", "lambda", "monad", "list", "string", "number", "table"
11   , "window", "garden", "music", "ocean", "forest", "cloud", "stone"
12   ]
13

```

```

14 genWord :: Gen String
15 genWord = elements dictionary
16
17 genSpaces :: Gen String
18 genSpaces = (`replicate` ' ') <$> choose (1, 3)
19
20 genPhrase :: Gen String
21 genPhrase = do
22     n <- choose (2, 6)
23     ws <- vectorOf n genWord
24     return (unwords ws)
25
26 genText :: Gen String
27 genText = concat <$> listOf1 (oneof [genWord, genSpaces])
28
29 prop_addMod_mod :: Int -> Int -> Property
30 prop_addMod_mod x y = forAll genMod $ \m ->
31     addMod x y m `mod` m == (x + y) `mod` m
32
33 prop_addMod_neutral :: Int -> Property
34 prop_addMod_neutral x = forAll genMod $ \m ->
35     addMod x 0 m == x `mod` m
36
37 prop_addMod_comm :: Int -> Int -> Property
38 prop_addMod_comm x y = forAll genMod $ \m ->
39     addMod x y m == addMod y x m
40
41 prop_rev_empty :: Bool
42 prop_rev_empty = reverseWords "" == ""
43
44 prop_rev_single :: Property
45 prop_rev_single = forAll genWord $ \w ->
46     reverseWords w == w
47
48 prop_rev_order :: Property
49 prop_rev_order = forAll genPhrase $ \s ->
50     words (reverseWords s) == reverse (words s)
51
52 prop_rev_involution :: Property
53 prop_rev_involution = forAll genText $ \s ->
54     reverseWords (reverseWords s) == s
55
56 main :: IO ()
57 main = do
58     putStrLn "addMod -- приведение по модулю:"
59     quickCheck (withMaxSuccess 50 prop_addMod_mod)
60     putStrLn "addMod -- нейтральный элемент:"
61     quickCheck (withMaxSuccess 50 prop_addMod_neutral)
62     putStrLn "addMod -- коммутативность:"
63     quickCheck (withMaxSuccess 50 prop_addMod_comm)
64     putStrLn "reverseWords -- пустая строка:"
65     quickCheck (withMaxSuccess 50 prop_rev_empty)
66     putStrLn "reverseWords -- одно слово:"
67     quickCheck (withMaxSuccess 50 prop_rev_single)
68     putStrLn "reverseWords -- смена порядка слов:"
69     quickCheck (withMaxSuccess 50 prop_rev_order)
70     putStrLn "reverseWords -- двойное применение:"
71     quickCheck (withMaxSuccess 50 prop_rev_involution)

```
